CMSC131: Java Foundations and Computer Systems

Chapter 1

By Naren Ayinala

May 2026

Contents

1 What Is a Computer? 3

1.1 Hardware and software . . . 3

1.2 An example of what happens inside a computer . . . 3

2 Why Do Computers Use Binary? 4

2.1 Decimal: the base-10 number system . . . 4

2.2 Counting in base 2 . . . 4

2.3 Converting decimal to binary . . . 4

2.4 Hexadecimal: the base-16 number system . . . 5

2.5 Comparing decimal, binary, and hexadecimal . . . 5

2.6 How text, pictures, and sound become bits . . . 6

3 Why Do We Need Programming Languages? 6

3.1 Levels of programming languages . . . 6

4 Programming Paradigms and Object-Oriented Programming 7

4.1 Classes and objects: an early picture . . . 7

5 What Is Java? 8

6 How Java Actually Works: JDK, JRE, JVM, and Bytecode 8

6.1 The JDK: Java Development Kit . . . 8

6.2 The JVM: Java Virtual Machine . . . 8

6.3 The JRE: Java Runtime Environment . . . 8

6.4 From source code to output: every step . . . 9

6.5 Compiler errors, runtime errors, and logic errors . . . 9

7 Understanding Your First Java Program **9**

8 Identifiers, Keywords, Comments, and Packages **10**

9 Variables: Giving Data a Name **10**

10 Java's Eight Primitive Types **11**

10.1 Primitive assignment versus reference assignment 11

11 Arithmetic, Division, and Expressions **12**

11.1 Increment and compound assignment 12

11.2 Constants with final 12

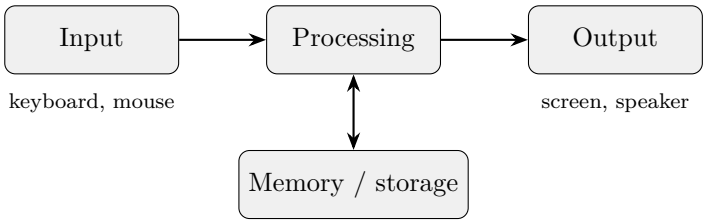
12 Putting It Together: A Complete Java Program **13**

13 Chapter Reference **13**

1 What Is a Computer?

A **computer** is an electronic machine that accepts information (input), processes it according to instructions, stores information, and produces results (output). A laptop, smartphone, game console, and server are all computers. The crucial idea is that a computer does not understand a problem the way a person does: it performs a sequence of precisely defined operations.

For example, when a student enters two exam scores into a grade calculator, the keyboard supplies the input, the processor executes the addition and division instructions, memory holds intermediate values, and the screen displays the average. The same general process happens when a computer loads a webpage or plays a video; only the instructions and data differ.



1.1 Hardware and software

Hardware is the physical equipment: the CPU, RAM, storage drive, motherboard, keyboard, and monitor. **Software** is the set of instructions and data that tell the hardware what to do. A Java program is software; the CPU executing its instructions is hardware.

The **CPU** (central processing unit) executes machine instructions. **RAM** (random-access memory) holds information that active programs need quickly. RAM is volatile: its contents are ordinarily lost when power is removed. **Persistent storage**, such as an SSD, retains files after shutdown. The operating system coordinates these resources and allows programs to run.

Component	Purpose	Example
CPU	Executes instructions and performs calculations	processor core
RAM	Fast working storage for running programs	16 GB RAM
SSD/HDD	Long-term storage for programs and files	512 GB SSD
Input device	Supplies information	keyboard
Output device	Presents results	monitor
Operating system	Manages resources and processes	Windows, macOS, Linux

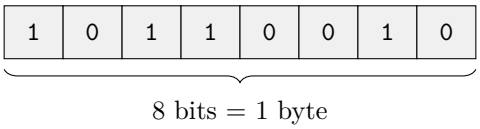
1.2 An example of what happens inside a computer

Suppose the user wants to calculate $7 + 5$. The input device captures the numbers. A program places the values in memory, the CPU executes an addition instruction, and the program sends the result 12 to the display. The CPU is not interpreting the mathematical meaning of the problem in English; it follows instructions that ultimately have machine-code representations.

2 Why Do Computers Use Binary?

Electronic circuits need a reliable way to represent information. In digital computers, two distinct signal states are especially convenient: a low and a high state, abstracted as **0** and **1**. These are **bits** (binary digits). A bit is not necessarily a literal switch, but the two-state model is how we reason about stored digital information.

Eight bits form a **byte**. With n bits there are 2^n possible patterns: one bit has 2 patterns, two bits have 4, three bits have 8, and eight bits have 256.



2.1 Decimal: the base-10 number system

The **decimal system** is the one we use in everyday life. It has ten possible digits, 0 through 9, and every position represents a power of ten. Moving one position left multiplies the place value by ten. For example,

$$3{,}472_{10} = 3(10^3) + 4(10^2) + 7(10^1) + 2(10^0) = 3000 + 400 + 70 + 2.$$

The subscript 10 means "base ten." When a number has no base indicated, we normally assume decimal. The important idea is *positional notation*: a digit's value depends on both the digit and where it appears.

Position	3	2	1	0
Place value	1000	100	10	1
Digit	3	4	7	2
Contribution	3000	400	70	2

2.2 Counting in base 2

In decimal, each digit represents a power of 10. In binary, each position represents a power of 2. The binary number 10110_2 is

$$1 \cdot 2^4 + 0 \cdot 2^3 + 1 \cdot 2^2 + 1 \cdot 2^1 + 0 \cdot 2^0 = 16 + 4 + 2 = 22_{10}.$$

The subscript tells us which base is being used. Without it, the written digits can be ambiguous.

Position	4	3	2	1	0
Power of two	16	8	4	2	1
Binary digit	1	0	1	1	0
Contribution	16	0	4	2	0

2.3 Converting decimal to binary

To convert 13 to binary, repeatedly divide by 2 and record the remainder. Read the remainders from bottom to top.

Number	Quotient after dividing by 2	Remainder
13	6	1
6	3	0
3	1	1
1	0	1

Therefore $13_{10} = 1101_2$. Check: $8 + 4 + 0 + 1 = 13$.

2.4 Hexadecimal: the base-16 number system

Hexadecimal (hex) is base sixteen. It uses the digits 0–9 and the letters A–F to represent values ten through fifteen. Programmers use hex because it is much shorter than writing long binary sequences. One hex digit represents **exactly four bits**, so two hex digits represent one byte.

Hex	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
Decimal	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

Just as decimal uses powers of ten and binary uses powers of two, hexadecimal uses powers of sixteen. For example,

$$2F_{16} = 2(16^1) + 15(16^0) = 32 + 15 = 47_{10}.$$

To convert binary to hex, divide the binary digits into groups of four, starting from the right. Add leading zeroes if needed:

$$\underbrace{1010}_{A}\ \underbrace{1111}_{F} \quad = \quad AF_{16} \quad = \quad 175_{10}.$$

For the reverse conversion, replace each hex digit with its four-bit binary equivalent: $3C_{16} = 0011\ 1100_2$. Hex is common in memory addresses, debugging, bit masks, and color notation. In Java, binary literals begin with `0b`, and hexadecimal literals begin with `0x`.

```
int decimal = 47; // base 10
int binary = 0b101111; // base 2: 32 + 8 + 4 + 2 + 1
int hex = 0x2F; // base 16: 2*16 + 15
System.out.println(decimal == binary); // true
System.out.println(binary == hex); // true
```

These three variables contain the same integer value. Only the way we wrote the number in the source code differs.

2.5 Comparing decimal, binary, and hexadecimal

Decimal (base 10)	Binary (base 2)	Hexadecimal (base 16)	Explanation
10	1010	A	$8 + 2$
15	1111	F	Four low bits set
16	10000	10	New hex place
26	11010	$1A$	$16 + 10$
47	101111	$2F$	$32 + 15$
255	11111111	FF	Maximum unsigned byte

Worked conversion: Convert 58_{10} into hexadecimal. Divide by 16: $58 = 3 \times 16 + 10$. The quotient is 3 and the remainder 10 is written A, so $58_{10} = 3A_{16}$. Convert to binary by expanding each hex digit: $3A_{16} = 0011\ 1010_2 = 111010_2$ after removing leading zeroes.

2.6 How text, pictures, and sound become bits

A computer stores more than numbers. Text is represented using character encodings such as Unicode; a character is associated with a numeric code point, and an encoding such as UTF-8 turns text into bytes. Images are often represented by pixels and color values, while digital sound is represented by sampled measurements. The file format determines how the bytes should be interpreted.

> **Important distinction:** binary is a representation of information. The pattern `01000001` could represent the number 65, the letter A in ASCII/UTF-8, or part of an image, depending on how the program interprets those bytes.

3 Why Do We Need Programming Languages?

A CPU ultimately executes **machine instructions**, encoded in binary. Writing a large application directly as machine-code bits would be extremely difficult for humans: even a small change could require carefully editing many instructions, and the result would be tied to a particular processor architecture.

A **programming language** lets us express instructions using readable names, expressions, control flow, and reusable structures. A translator (compiler or interpreter, depending on the language and implementation) bridges the gap between human-written code and machine execution.



3.1 Levels of programming languages

Machine language is the instruction encoding understood by a particular CPU. **Assembly language** uses symbolic names for those instructions, such as `ADD` or `MOV`; an assembler converts it to machine code. **High-level languages** provide abstractions such as methods, classes, lists, and loops, so we can describe a task without manually managing every processor instruction.

Language	Common use	Typical style
C	Systems software, embedded systems	procedural
C++	Games, systems, performance-intensive apps	multi-paradigm
Java	Enterprise apps, backend services, Android-related code	object-oriented
Python	Scripting, automation, data science	multi-paradigm
JavaScript	Web interfaces and server applications	multi-paradigm
C#	Applications and services on .NET	object-oriented
Rust	Systems programming with memory safety	multi-paradigm
SQL	Querying and manipulating relational data	declarative

There is no single universally best language. Languages offer different trade-offs in readability, speed, portability, tooling, and the kinds of programs they make convenient to write.

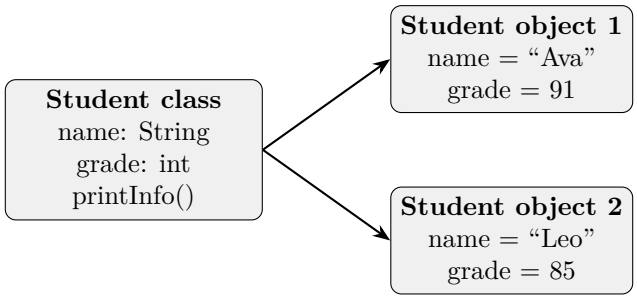
4 Programming Paradigms and Object-Oriented Programming

A **programming paradigm** is a way of organizing programs and expressing computation. Procedural programming organizes code around procedures or functions. Functional programming emphasizes computing results through functions and values. **Object-oriented programming (OOP)** organizes programs around objects that combine state and behavior.

In CMSC131, we will learn programming primarily through **Java**, which supports object-oriented programming. This means we will gradually learn how to create classes, construct objects, store information in fields, and define methods that operate on that information.

4.1 Classes and objects: an early picture

Think of a **class** as a description or blueprint and an **object** as an individual instance created from that description. For example, a `Student` class might describe a name and a grade; two student objects can hold different values while following the same class definition.




```
class Student {
    String name;
    int grade;

    void printInfo() {
        System.out.println(name + ": " + grade);
    }
}

public class School {
    public static void main(String[] args) {
        Student first = new Student();
        first.name = "Ava";
        first.grade = 91;
        first.printInfo(); // Ava: 91
    }
}
```

The class describes which fields and methods student objects have. The expression `new Student()` creates an object; the variable `first` holds a reference to it. We will study references, constructors, and encapsulation in later chapters.

5 What Is Java?

Java is a general-purpose, statically typed programming language. **Statically typed** means that Java checks types at compile time; for example, assigning a string to an `int` variable is a compile-time error. Java's syntax is case-sensitive, so `main`, `Main`, and `MAIN` are different identifiers.

One of Java's goals is portability: instead of compiling ordinary Java source directly into a different native executable for each CPU, the Java compiler usually produces **bytecode**, which a compatible Java Virtual Machine can execute on different platforms. This is often summarized as "write once, run anywhere," although programs that rely on platform-specific resources may still need changes.

6 How Java Actually Works: JDK, JRE, JVM, and Bytecode

These terms describe different layers of the Java development and execution environment. They are related, but they are *not* interchangeable.

6.1 The JDK: Java Development Kit

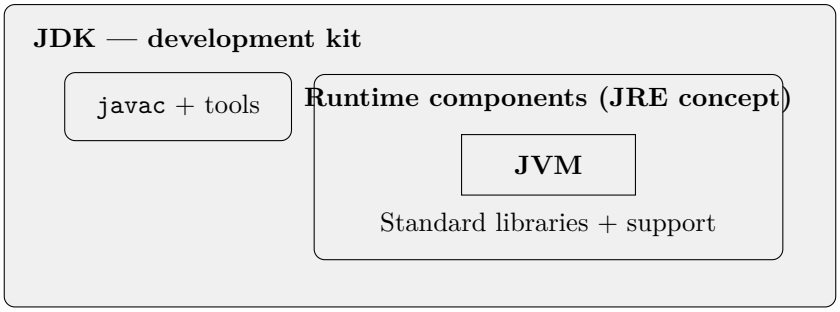
The **JDK** is what a developer installs to build Java programs. It includes the Java compiler (`javac`), tools, libraries, and the components needed to run Java programs. When writing Java for CMSC131, a JDK is the usual choice because we need both to *compile* and to *run* code.

6.2 The JVM: Java Virtual Machine

The **JVM** is the execution engine that runs Java bytecode. It loads classes, verifies bytecode, manages memory, performs garbage collection, and executes instructions. JVM implementations may interpret bytecode and use **just-in-time (JIT) compilation** to translate frequently executed code into optimized native machine instructions.

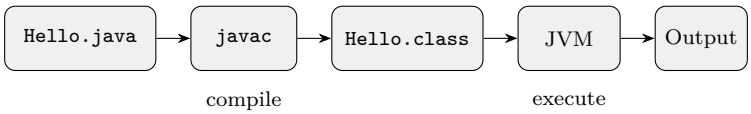
6.3 The JRE: Java Runtime Environment

The **JRE** refers to the environment needed to *run* Java programs: the JVM plus the supporting libraries and runtime components. Historically it was often installed separately from the JDK. In modern Java releases, a separate public JRE download is not always provided; a JDK contains runtime capabilities, and custom runtime images can be built. Conceptually, the distinction remains useful: *development tools versus runtime environment versus execution engine.*



6.4 From source code to output: every step

Consider a file called `Hello.java`. First, we write source code. Next, `javac` checks the code and creates `Hello.class`, which contains bytecode. Finally, the `java` launcher starts a JVM, loads the class, and executes its `main` method.




```
public class Hello {
    public static void main(String[] args) {
        System.out.println("Hello, world!");
    }
}
```

Terminal commands:

```
javac Hello.java
java Hello
```

The first command creates the class file (if compilation succeeds). The second runs it. Notice that we type `java Hello`, not `java Hello.class`. The output is `Hello, world!`.

6.5 Compiler errors, runtime errors, and logic errors

A **compile-time error** means the compiler cannot translate the program successfully, such as a missing semicolon or incompatible type. A **runtime error** occurs while the program executes, such as dividing an integer by zero. A **logic error** means the program runs but calculates the wrong result.

```
int age = "twenty"; // Compile-time type error
int result = 10 / 0; // Runtime ArithmeticException
int average = (80 + 90) / 3; // Runs, but wrong if averaging 2 scores
```

> **Java source, bytecode, and machine code are different.** Source is what we write. Bytecode is what `javac` normally produces. The JVM executes bytecode, potentially using native machine code generated by a JIT compiler.

7 Understanding Your First Java Program

```
public class Hello {
    public static void main(String[] args) {
        System.out.println("Hello, CMSC131!");
    }
}
```

`public class Hello` declares a class named `Hello`. In this example, the source file is named `Hello.java`; a public top-level class must be declared in a correspondingly named source file. The braces `{` and `}` delimit the class body.

`public static void main(String[] args)` is the conventional entry point for introductory Java programs. `public` permits access, `static` means the method belongs to the class rather than requiring an object, `void` means it does not return a value, and `String[] args` holds command-line arguments. We will unpack methods and arrays later.

`System.out.println(...)` writes text to standard output and then starts a new line. `System.out.print(...)` writes without adding that final newline. A semicolon ends a Java statement, and double quotes delimit a string literal.

```
System.out.print("Hello ");
System.out.println("there!");
System.out.println(3 + 4);
// Output:
// Hello there!
// 7
```

8 Identifiers, Keywords, Comments, and Packages

An **identifier** is a name chosen for a variable, method, class, or other program element. Names are case-sensitive. Java identifiers can contain letters, digits, `_`, and `$`, but cannot begin with a digit or be reserved keywords. In normal Java style, class names use `UpperCamelCase`, and variables and methods use `lowerCamelCase`.

Identifier	Legal?	Reason
`studentCount`	Yes	Begins with a letter
`_total`	Yes	Underscore followed by letters
`2students`	No	Begins with a digit
`class`	No	Reserved keyword
`first-name`	No	Hyphen is not allowed
`Student` vs `student`	Both	Different names due to case sensitivity

Comments explain code for human readers and are ignored by the compiler as executable instructions. Java has single-line comments (`//`), block comments (`/* ... */`), and documentation comments (`/** ... */`). A **package** groups related classes under a namespace. A package declaration, if present, appears before imports and class declarations.

```
package school;

/** Represents a simple course. */
public class Course {
    // Number of students currently enrolled
    int enrollment = 30;
}
```

9 Variables: Giving Data a Name

A **variable** is a named storage location with a declared type. Declaring a variable tells Java what kind of value it can hold. Initializing it gives it a starting value. An assignment updates the value.

```
int score; // declaration
score = 85; // first assignment
score = score + 5; // update: score is now 90
System.out.println(score); // 90
```

After statement	score	Explanation
`int score;`	uninitialized	No local value assigned yet
`score = 85;`	85	Store 85
`score = score + 5;`	90	Read 85, add 5, store 90

The right-hand side of an assignment is evaluated first. Then its resulting value is stored in the variable on the left. The equals sign `=` means *assignment*, not a mathematical statement of equality.

10 Java's Eight Primitive Types

Java has eight primitive types. They directly represent simple values rather than objects. Choosing a type tells Java which operations and ranges are appropriate.

Type	Typical size	Example / meaning
`byte`	8 bits	Signed integer, −128 to 127
`short`	16 bits	Signed integer
`int`	32 bits	Common whole-number type
`long`	64 bits	Larger whole numbers, e.g. `100L`
`float`	32 bits	Floating point, e.g. `2.5f`
`double`	64 bits	Common decimal approximation
`char`	16 bits	One UTF-16 code unit, e.g. `'A'`
`boolean`	JVM-dependent	`true` or `false`

A `String` is *not* a ninth primitive. It is a class, and a `String` variable holds a reference to a string object. The type `char` uses single quotes for a character literal; `String` literals use double quotes.

```
int year = 2026;
double price = 19.95;
boolean enrolled = true;
char letter = 'A';
String message = "Welcome";
```

10.1 Primitive assignment versus reference assignment

When you assign one primitive variable to another, Java copies the *value*. Changing the second variable does not change the first.

```
int x = 5;
int y = x;
y = 12;
System.out.println(x); // 5
System.out.println(y); // 12
```



Two separate primitive values

References behave differently because the copied value is a reference to an object. We will explore object diagrams in the classes chapter.

11 Arithmetic, Division, and Expressions

Java provides `+`, `-`, `*`, `/`, and `%` (remainder). With two integer operands, division discards the fractional part by truncating toward zero. With a floating-point operand, division produces a floating-point result.

```
System.out.println(7 / 2); // 3
System.out.println(7 % 2); // 1
System.out.println(7 / 2.0); // 3.5
System.out.println(7.0 / 2); // 3.5
```

Order of operations: multiplication, division, and remainder bind more tightly than addition and subtraction. Parentheses explicitly control grouping. For operators at the same precedence level, arithmetic expressions are generally evaluated left to right.

```
int a = 2 + 3 * 4; // 14
int b = (2 + 3) * 4; // 20
int c = 17 / 5 * 2; // (3) * 2 = 6
```

11.1 Increment and compound assignment

`x += 3` updates `x` by adding 3. `x++` increases `x` by 1, but as an expression it produces the old value; `++x` increases it first and produces the new value.

```
int x = 4;
int a = x++; // a = 4, x = 5
int b = ++x; // b = 6, x = 6
x += 3; // x = 9
```

Statement completed	x	a	b
`int x = 4;`	4	–	–
`int a = x++;`	5	4	–
`int b = ++x;`	6	4	6
`x += 3;`	9	4	6

11.2 Constants with final

A `final` variable can be assigned only once. Constants are often written in uppercase with underscores, especially when declared `static final`.

```
final double SALES_TAX = 0.06;
double total = 50 * (1 + SALES_TAX); // 53.0
```

12 Putting It Together: A Complete Java Program

Here is a small program that uses variables, arithmetic, a class, and printed output. Trace each line to see how the result is computed.

```
public class GradeCalculator {
    public static void main(String[] args) {
        int exam1 = 84;
        int exam2 = 92;
        int total = exam1 + exam2;
        double average = total / 2.0;

        System.out.println("Total: " + total);
        System.out.println("Average: " + average);
    }
}
// Total: 176
// Average: 88.0
```

The expression `total / 2.0` is deliberate: `2.0` is a `double`, so Java performs floating-point division. If the scores summed to an odd number, using `2` instead could silently discard the fractional part before assigning to `average`.

13 Chapter Reference

Term	Meaning
Hardware	Physical components of a computer
Software	Instructions and data used by hardware
CPU	Executes machine instructions
RAM	Fast volatile working memory
Bit / byte	One binary digit / eight bits
Programming language	Human-readable way to express instructions
OOP	Organizes programs using classes and objects
JDK	Java development tools plus runtime components
JRE	Java runtime environment concept
JVM	Engine that executes Java bytecode
`javac`	Compiles `.java` source into `.class` bytecode
`java`	Launches a Java application
Primitive	Built-in value type such as `int` or `boolean`